

Système automatisé de gestion de jardin potager

Raspberry Pi 5 · Arduino Edge Control · Flask · SQLite

Auteur	Patrick Pinard
Version	1.0
Date	18 April 2026
Localisation	Vullierens, Vaud, Suisse
Firmware	1.0.0 (PlatformIO / Arduino Edge Control)
Backend	Python 3.10 · Flask 3.x · APScheduler

Table des matières

1	Architecture du système	3
2	Matériel requis	3
3	Montage — Capteurs sol (SoilWatch 10)	4
4	Montage — Capteurs température DS18B20	5
5	Montage — Anémomètre	6
6	Montage — Vannes 24V latching	6
7	Montage — Vérin linéaire (lucarne)	7
8	Alimentation	8
9	Firmware Arduino — Configuration	8
10	API REST Arduino ↔ Raspberry Pi	9
11	Bascule Simulation → Production	12
12	Checklist de mise en service	13

1. Architecture du système

MonJardin est un système en deux couches communicantes via WiFi. Le **Raspberry Pi 5** exécute le serveur Flask, le moteur de décision (arrosage, toit, météo) et l'interface web PWA. L'**Arduino Edge Control** gère exclusivement le hardware : acquisition des capteurs et pilotage des actionneurs. La communication est unidirectionnelle : le Raspberry interroge l'Arduino via API REST JSON toutes les 60 secondes.

Couche	Matériel	Rôle
Décision	Raspberry Pi 5	Flask · APScheduler · SQLite · Météo Open-Meteo · Interface PWA
Terrain	Arduino Edge Control	Lecture capteurs · Pilotage vannes · Contrôle vérin · Serveur HTTP
Capteurs sol	SoilWatch 10 ×4	Humidité volumétrique par zone (ADC 16-bit)
Température	DS18B20 ×2	Température extérieure + intérieure serre (OneWire)
Vent	Anémomètre QS-FS01	Vitesse vent en km/h (sortie tension analogique 0.4–2.0 V)
Irrigation	Vannes 24V latching ×4	Ouverture/fermeture par zone (pulse relais)
Lucarne	Vérin linéaire 12V	Ouverture toit serre (H-bridge + fins de course)

2. Matériel requis

Référence	Quantité	Rôle	Interface Arduino
Arduino Edge Control	1	Contrôleur principal terrain	—
Raspberry Pi 5 (4 GB)	1	Serveur Flask / décision	WiFi 802.11ac
SoilWatch 10	4	Capteur humidité sol 0–100%	InputExpander ADC
DS18B20 (sonde étanche)	2	Température ext. + serre	OneWire D5
Anémomètre QS-FS01	1	Vitesse vent (tension 0.4–2.0 V)	InputExpander ADC 4
Vanne solénoïde 24V DC latching	4	Irrigation par zone	Relay 0–3
Vérin linéaire 12V	1	Ouverture lucarne serre	GPIO D7/D8 + D9/D10
Alimentation 24V / 5A	1	Vannes latching	—
Alimentation 12V / 2A	1	Vérin linéaire	—
Alimentation 5V / 5A USB-C	1	Raspberry Pi 5	—
Boîtier IP67	1	Protection intempéries	—
Câble 2×0.75mm ² (rouge/noir)	~20m	Alimentation capteurs/actionneurs	—

3. Montage — Capteurs sol SoilWatch 10

Les capteurs SoilWatch 10 mesurent l'humidité volumétrique du sol par résistivité électrique et fournissent une tension analogique de **0 V (sec)** à **3 V (saturé)**. L'Arduino Edge Control les lit via son **InputExpander** avec ADC 16-bit.

Connexion physique

Zone	Canal InputExpander	Fil Signal	Fil GND	Alimentation
Zone 1 — Serre	Canal 0 (AI0)	Jaune/Blanc	Noir	3.3V (rouge)
Zone 2 — Potager	Canal 1 (AI1)	Jaune/Blanc	Noir	3.3V (rouge)
Zone 3 — Mi-ombre	Canal 2 (AI2)	Jaune/Blanc	Noir	3.3V (rouge)
Zone 4 — Aromates	Canal 3 (AI3)	Jaune/Blanc	Noir	3.3V (rouge)

Méthode de lecture (firmware)

L'InputExpander de l'Edge Control remplace les broches ADC classiques. Il est initialisé avec **Expander.begin()**. Chaque lecture est la moyenne de **8 échantillons** (ADC_SAMPLES) espacés de 5 ms pour filtrer le bruit. La conversion ADC→% utilise une interpolation linéaire entre deux valeurs de calibration :

```
humidité (%) = 100 × (ADC_DRY - ADC_mesuré) / (ADC_DRY - ADC_WET)
```

Calibration

Paramètre	Valeur par défaut	Signification
ADC_DRY	3100	Valeur ADC lorsque le capteur est dans l'air sec
ADC_WET	1200	Valeur ADC lorsque le capteur est dans l'eau

■ ■ Ces valeurs sont globales (même calibration pour les 4 zones). Pour une précision maximale, calibrer chaque capteur individuellement avec `setCalibration(zone_id, dry, wet)` dans `main.cpp`.

Notes d'installation

- Insérer le capteur verticalement dans le sol à **10–15 cm** de profondeur, au niveau des racines actives.
- Ne pas plier le câble à moins de **5 cm** du boîtier.
- Protéger les connecteurs avec du ruban auto-amalgamant ou une gaine thermo.
- Le capteur fonctionne dans des sols de conductivité EC < 2 mS/cm (sols standard non-fertilisés intensivement).

4. Montage — Capteurs température DS18B20

Les DS18B20 communiquent via le protocole **1-Wire (OneWire)** sur une seule broche de données. Les deux capteurs partagent le **même bus** (broche D5). Chaque capteur possède un identifiant 64-bit unique gravé en usine.

Connexion physique — bus partagé

Capteur	Pin D5 (Data)	GND	VCC	Résistance pull-up
DS18B20 #0 — Extérieur	Fil jaune	Noir	Rouge (3.3V)	4.7 kΩ entre Data et VCC
DS18B20 #1 — Serre intérieure	Fil jaune	Noir	Rouge (3.3V)	(une seule résistance sur le bus)

■■ Important : La résistance pull-up 4.7 kΩ est obligatoire entre la broche Data et VCC. Sans elle, les lectures retournent **−127°C**.

Méthode de lecture (firmware)

La bibliothèque **DallasTemperature** gère l'adressage OneWire automatiquement. L'ordre des capteurs dépend de leur ordre de découverte sur le bus (`setWaitForConversion(false)` pour mode asynchrone) :

- Index 0 → `getExterior()` → température extérieure
- Index 1 → `getGreenhouse()` → température intérieure serre

■■ Si les deux capteurs sont inversés (serre affiche température extérieure), inverser physiquement les capteurs ou changer les index dans `TempSensor.cpp`.

Lecture toutes les **30 secondes** (`TEMP_READ_INTERVAL = 30 000 ms`). En cas de lecture invalide (−127°C ou 85°C), la dernière valeur connue est conservée.

5. Montage — Anémomètre QS-FS01

Le QS-FS01 est un anémomètre à **sortie tension analogique** (0.4 V–2.0 V). La tension est convertie en vitesse de vent selon la formule du fabricant. Il n'utilise **pas** de sortie impulsions — il est connecté sur le canal ADC 4 de l'InputExpander de l'Arduino Edge Control.

Caractéristiques électriques

Paramètre	Valeur	Description
Alimentation	7–24 V DC	Ne pas alimenter en 3.3 V — tension minimale 7 V
Sortie signal	0.4 V à 2.0 V	Analogique — 0.4 V = vent nul, 2.0 V = 32.4 m/s
Plage de mesure	0–32.4 m/s	Soit 0–116.6 km/h
Formule de conversion	$(V - 0.4) / 1.6 \times 32.4$	Résultat en m/s · multiplier par 3.6 pour km/h
Canal ADC	InputExpander 4	Canaux 0–3 réservés aux SoilWatch
Lectures moyennées	8	WIND_ADC_SAMPLES — réduction du bruit
Période de lecture	5 s	lastWindReadMs — géré dans loop()

Câblage

Fil QS-FS01	Couleur typique	Connexion Arduino Edge Control
Alimentation	Rouge / Marron	Bornier 12V ou 24V DC (7 V min)
Masse	Bleu / Noir	GND commun Arduino
Signal	Jaune / Bleu clair	InputExpander ADC canal 4

■ ■ L'alimentation doit être ≥ 7 V DC. Ne jamais alimenter le QS-FS01 depuis la pin 3.3 V de l'Arduino — la tension est insuffisante et le signal restera à 0.4 V (vent nul) en permanence.

Formule complète (implémentée dans AnemometerSensor.cpp)

```
int avgAdc = moyenne(8 × InputExpander.analogRead(4)); float V = avgAdc × (3.3 / 65535.0);  
// ADC 16-bit → tension float ms = (V - 0.4) / 1.6 × 32.4; // formule datasheet float kmh  
= ms × 3.6; // m/s → km/h
```

6. Montage — Vannes 24V latching

Les vannes solénoïdes **latching** (bistables) maintiennent leur position (ouverte ou fermée) sans courant continu. Seule une **impulsion courte** de **50 ms** est nécessaire pour changer d'état, ce qui minimise la consommation.

Connexion aux relais Edge Control

Zone	Relais Edge Control	Bobine +	Bobine –	Alimentation
Zone 1 — Serre	Relay 0	Bobine set	Bobine reset	24V DC / ~500 mA par ouverture
Zone 2 — Potager	Relay 1	Bobine set	Bobine reset	24V DC / ~500 mA par ouverture
Zone 3 — Mi-ombre	Relay 2	Bobine set	Bobine reset	24V DC / ~500 mA par ouverture
Zone 4 — Aromates	Relay 3	Bobine set	Bobine reset	24V DC / ~500 mA par ouverture

■ ■ Bug connu (v1.0) : Le firmware ValveController.cpp envoie actuellement la même impulsion pour ouvrir ET fermer. Pour les relais latching avec deux bobines distinctes (set/reset), il faut utiliser deux index de relais par vanne (ex. Relay 0 = set, Relay 4 = reset pour Zone 1). Corriger dans ValveController::_pulse() avant déploiement réel.

Séquence d'initialisation

Au démarrage du firmware (begin()), toutes les vannes reçoivent une impulsion de fermeture (sécurité). Même si l'état interne indique 'fermé', l'impulsion est envoyée pour garantir la position physique après une coupure de courant.

Protection contre l'arrosage excessif (firmware)

Paramètre	Valeur	Description
IRRIGATION_MAX_PER_HOUR	4	Max 4 déclenchements par zone par heure
IRRIGATION_MIN_INTERVAL_MS	300 000 ms (5 min)	Délai minimum entre deux arrosages
IRRIGATION_ALERT_COOLDOWN	3 600 000 ms (1h)	Délai entre deux alertes email

7. Montage — Vérin linéaire (lucarne serre)

Le vérin linéaire 12V est commandé par un **pont en H (H-bridge)** via deux sorties GPIO de l'Arduino (D7 et D8). Deux **fins de course** (D9 et D10) signalent les positions extrêmes et coupent le moteur automatiquement.

Signal	Broche	Type	Rôle
IN1 (direction A)	D7	OUTPUT	Sens extension (ouverture)
IN2 (direction B)	D8	OUTPUT	Sens rétraction (fermeture)
ENDSTOP_OPEN	D9	INPUT_PULLUP	Fin de course position ouverte
ENDSTOP_CLOSE	D10	INPUT_PULLUP	Fin de course position fermée

Logique de commande

Action	IN1	IN2	Résultat
Ouverture	HIGH	LOW	Vérin s'étend — lucarne s'ouvre
Fermeture	LOW	HIGH	Vérin se rétracte — lucarne se ferme
Arrêt	LOW	LOW	Moteur coupé (frein)

Sécurités

- **Timeout 60 s** (ACTUATOR_TIMEOUT_MS) : si la fin de course n'est pas atteinte en 60 secondes, le moteur est coupé et l'état passe en ERROR.
- Les fins de course sont câblés en **Normalement Fermé (NC)** avec pull-up interne → l'ouverture du contact (HIGH) signale l'atteinte de la position.
- La méthode update() doit être appelée dans chaque itération du loop() pour surveiller les fins de course en temps réel.

8. Alimentation

Composant	Tension	Courant max	Notes
Vannes latching	24V DC	500 mA × 4 = 2A (simultané)	Alimentation dédiée, masse commune avec Arduino
Vérin linéaire	12V DC	2A	Alimentation dédiée
Arduino Edge Control	7–30V	500 mA	Peut être alimenté depuis le 24V
Raspberry Pi 5	5V DC	5A	USB-C, alimentation officielle RPi
Capteurs SoilWatch 10	3.3V	50 mA total	Fourni par Arduino via pin 3.3V
DS18B20	3.3V	1.5 mA chacun	Fourni par Arduino via pin 3.3V
Anémomètre QS-FS01	7–24V DC	< 20 mA	Alimentation dédiée ≥ 7 V requise

■ ■ Ne jamais connecter la masse 24V directement à la masse 3.3V de l'Arduino sans isolateur optique. Les relais de l'Edge Control assurent déjà l'isolation galvanique.

9. Firmware Arduino — Configuration

Tous les paramètres modifiables sont centralisés dans `arduino_edge_control/src/config.h`. Modifier ce fichier puis recompiler avec PlatformIO avant de flasher.

Paramètres à configurer avant déploiement

Constante	Valeur défaut	À modifier
WIFI_SSID	"MonReseau"	SSID de votre réseau WiFi
WIFI_PASSWORD	"MotDePasse"	Mot de passe WiFi
RPI_HOST	"192.168.1.10"	Adresse IP fixe du Raspberry Pi
RPI_PORT	5001	Port Flask (défaut 5001)
ADC_DRY	3100	Valeur ADC capteur sol à sec (calibrer)
ADC_WET	1200	Valeur ADC capteur sol saturé (calibrer)
ANEMOMETER_ADC_CH	4	Canal InputExpander (0–3 = SoilWatch, 4 = QS-FS01)
WIND_V_ZERO	0.4f	Tension sortie à vent nul (V) — datasheet QS-FS01
WIND_V_FULL	2.0f	Tension sortie à vitesse max (V) — datasheet QS-FS01
WIND_MS_MAX	32.4f	Vitesse max correspondante (m/s) — datasheet QS-FS01
WIND_ADC_SAMPLES	8	Nombre de lectures ADC moyennées par mesure

Compilation et flash (PlatformIO)

```
cd arduino_edge_control pio run --target upload --environment edge_control
```

Surveillance série

```
pio device monitor --baud 115200
```

Les logs apparaissent au format : `[NIVEAU] [MODULE] message`. Exemple : `[INFO] [SOIL] Zone 1 ADC=2507 → 61.2%`

10. API REST Arduino ↔ Raspberry Pi

La communication est initiée **exclusivement par le Raspberry Pi**. L'Arduino expose un serveur HTTP léger sur le port **80**. Toutes les réponses sont en **JSON**, encodage UTF-8. Base URL : **http://<IP_ARDUINO>/api**

10.1 GET /api/sensors

Retourne les lectures de tous les capteurs en une seule requête. Appelé par le Raspberry Pi à chaque cycle d'automatisation (toutes les 60 s). C'est l'endpoint le plus utilisé — il concentre humidité sol, températures et vent.

Réponse JSON :

```
{ "temperature_c": 26.0, "temp_serre_c": 38.9, "wind_speed_kmh": 16.0, "zones": [ {  
  "zone_id": 1, "soil_moisture_pct": 61.2, "raw_adc": 2507, "valid": true }, { "zone_id": 2,  
  "soil_moisture_pct": 67.6, "raw_adc": 2769, "valid": true }, { "zone_id": 3,  
  "soil_moisture_pct": 53.7, "raw_adc": 2197, "valid": true }, { "zone_id": 4,  
  "soil_moisture_pct": 39.4, "raw_adc": 1615, "valid": true } ] }
```

Champ	Type	Description
temperature_c	float	Température extérieure DS18B20 #0, en °C
temp_serre_c	float	Température intérieure serre DS18B20 #1, en °C
wind_speed_kmh	float	Vitesse du vent calculée sur 3s, en km/h
zones[].zone_id	int	Identifiant de zone (1–4)
zones[].soil_moisture_pct	float	Humidité volumétrique 0–100% après calibration
zones[].raw_adc	int	Valeur brute ADC 0–4095 (utile pour calibration)
zones[].valid	bool	False si le capteur est hors-ligne ou déconnecté

10.2 GET /api/actuators/status

Retourne l'état actuel de tous les actionneurs : vannes d'irrigation et lucarne de serre. Le Raspberry Pi appelle cet endpoint après chaque commande pour confirmer l'exécution.

Réponse JSON :

```
{ "roof_state": "open", "valves": [ { "zone_id": 1, "state": "close" }, { "zone_id": 2,  
  "state": "open" }, { "zone_id": 3, "state": "close" }, { "zone_id": 4, "state": "close" }  
] }
```

Champ	Valeurs possibles	Description
roof_state	open · close · moving_open · moving_close · error	État du vérin lucarne. 'moving_*' = en cours de déplacement
valves[].state	open · close	État de la vanne pour cette zone

10.3 POST /api/actuators/valve/<zone_id>

Commande l'ouverture ou la fermeture d'une vanne d'irrigation. Le paramètre **zone_id** est passé dans l'URL (1–4). L'Arduino envoie une impulsion de 50 ms sur le relais correspondant.

Corps de la requête :

```
{ "state": "open" } // ou "close"
```

Champ	Valeurs	Obligatoire	Description
state	"open" · "close"	Oui	État désiré pour la vanne

Réponse succès :

```
{ "ok": true }
```

Réponse erreur (400) :

```
{ "ok": false, "error": "state doit etre 'open' ou 'close'" }
```

10.4 POST /api/actuators/roof

Commande l'ouverture ou la fermeture de la lucarne de serre via le vérin linéaire. Le moteur s'arrête automatiquement à la détection du fin de course (max 60 s). Si la fin de course n'est pas atteinte, l'état passe à 'error'.

Corps de la requête :

```
{ "state": "open" } // ou "close"
```

Réponse :

```
{ "ok": true }
```

L'état réel du vérin est accessible via GET /api/actuators/status (roof_state). La valeur 'moving_open' ou 'moving_close' indique que le déplacement est en cours.

10.5 GET /api/health

Endpoint de supervision. Le Raspberry Pi l'appelle périodiquement pour vérifier que l'Arduino est joignable et fonctionnel. En mode simulation, cet endpoint est également exposé par l'émulateur sur le port 8081.

Réponse JSON :

```
{ "status": "ok", "firmware_version": "1.0.0", "uptime_s": 2847, "wifi_rssi": -65 }
```

Champ	Type	Description
status	string	'ok' si le firmware fonctionne normalement
firmware_version	string	Version du firmware (config.h FIRMWARE_VERSION)
uptime_s	int	Durée de fonctionnement depuis le dernier reset, en secondes
wifi_rssi	int	Force du signal WiFi en dBm (idéal > -70 dBm)

Récapitulatif des endpoints

Endpoint	Méthode	Fréquence appel	Rôle
/api/sensors	GET	60 s (automatisme)	Lecture tous capteurs
/api/actuators/status	GET	À la demande	État vannes + lucarne
/api/actuators/valve/<id>	POST	À la demande	Commande vanne
/api/actuators/roof	POST	À la demande	Commande lucarne
/api/health	GET	5 min (supervision)	État Arduino + WiFi

11. Bascule Simulation → Production

Le système bascule intégralement en production en modifiant uniquement le fichier **garden_manager/.env** et en redémarrant Flask. Aucune modification de code n'est nécessaire.

Modifications dans .env

```
SIMULATION_MODE=false ARDUINO_API_URL=http://192.168.1.XXX:80/api FLASK_DEBUG=false
```

En mode production :

- L'émulateur Arduino (port 8081) ne démarre **pas**.
- ArduinoClient appelle directement l'IP physique de l'Arduino.
- Les profils météo simulés sont désactivés dans les Paramètres.
- L'accélérateur de simulation (SIMULATION_SPEED) n'a aucun effet sur les intervalles.

Redémarrage sur Raspberry Pi

```
git pull origin main sudo systemctl restart monjardin.service
```

12. Checklist de mise en service

Firmware Arduino

✓	Étape	Référence
■	Mettre à jour WIFI_SSID et WIFI_PASSWORD dans config.h	config.h:4-5
■	Mettre à jour RPI_HOST avec l'IP fixe du Raspberry Pi	config.h:9
■	Calibrer ADC_DRY et ADC_WET pour vos capteurs SoilWatch	config.h:26-27
■	Vérifier câblage QS-FS01 : alimentation $\geq 7V$, signal sur ADC 4	config.h, AnemometerSensor.cpp
■	Corriger ValveController::_pulse() pour relais latching	ValveController.cpp:50
■	Compiler et flasher via PlatformIO	platformio.ini
■	Vérifier logs série : 4 zones ADC, 2 DS18B20 détectés	Serial 115200
■	Tester GET http://api/health depuis le Pi	RestServer

Raspberry Pi / Flask

✓	Étape	Référence
■	Mettre une IP fixe au Raspberry Pi sur le réseau local	dhcpcd.conf
■	Modifier .env : SIMULATION_MODE=false	.env
■	Modifier .env : ARDUINO_API_URL=http://:80/api	.env
■	Modifier .env : FLASK_DEBUG=false	.env
■	Tester GET /api/health depuis l'interface web	Page Arduino
■	Vérifier lecture humidité toutes les 60 s en DB	Page Journal
■	Tester arrosage manuel Zone 1 → vérifier ouverture physique	Dashboard
■	Tester fermeture vanne → vérifier fermeture physique	Dashboard
■	Tester ouverture/fermeture lucarne serre	Dashboard
■	Vérifier alertes email si humidité < seuil	Paramètres